



中国科学技术大学

University of Science and Technology of China

计算系统概论A

Introduction to Computing Systems
(CS1002A.03)

Chapter 7

Assembly Language Program

陈俊仕

cjuns@ustc.edu.cn

2025 Fall

计算机科学与技术学院

School of Computer Science and Technology

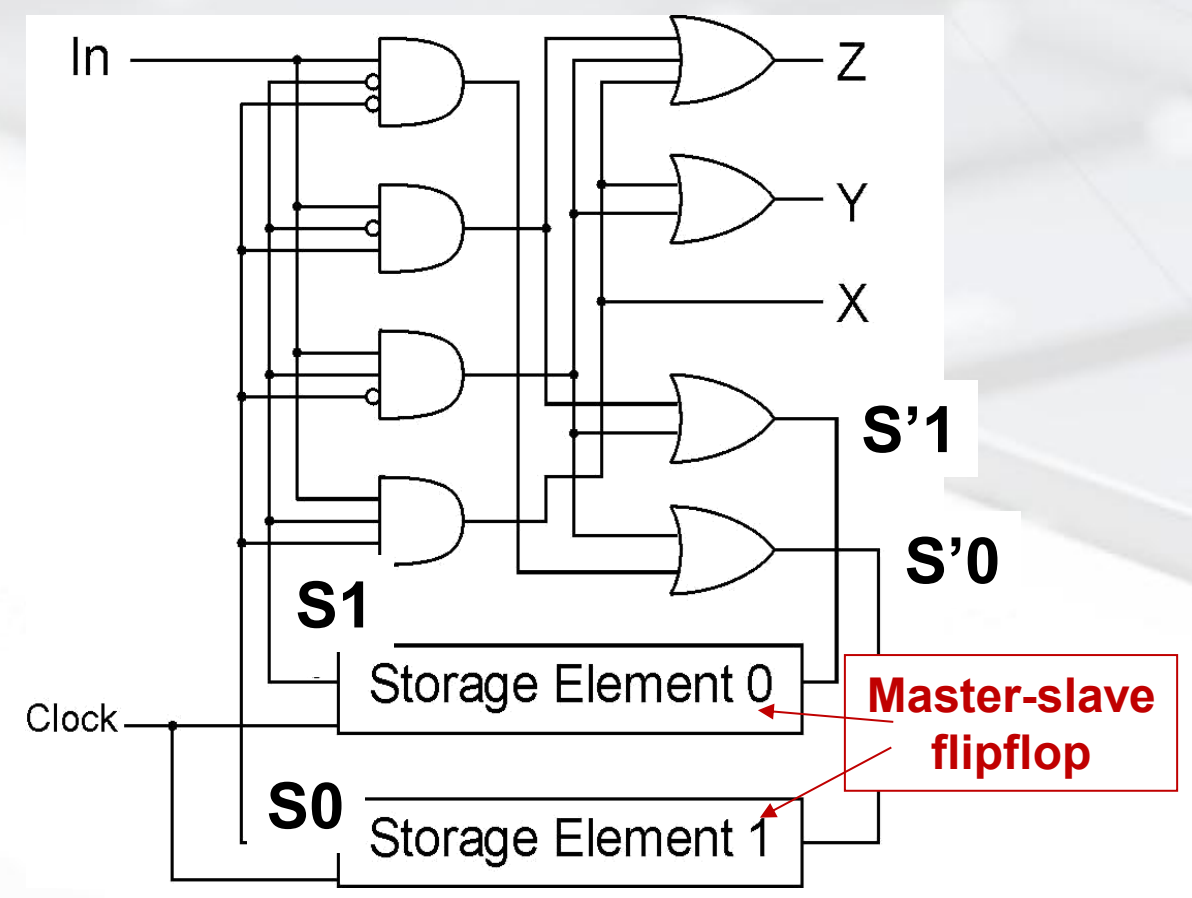
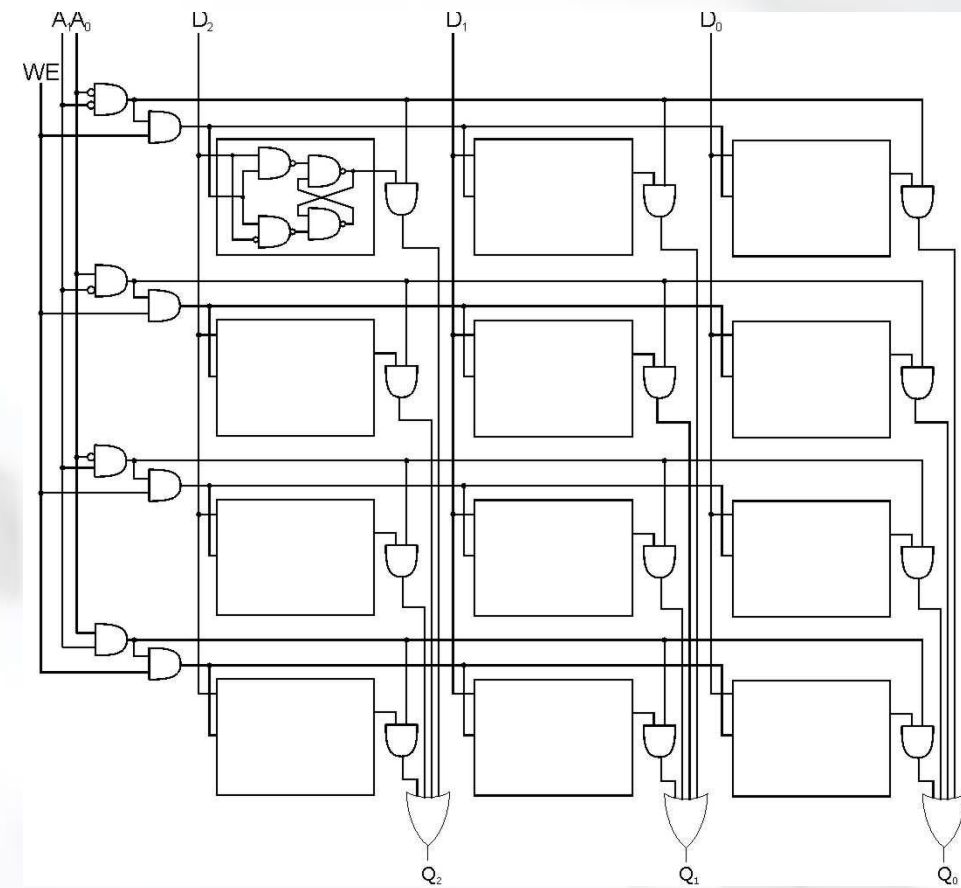
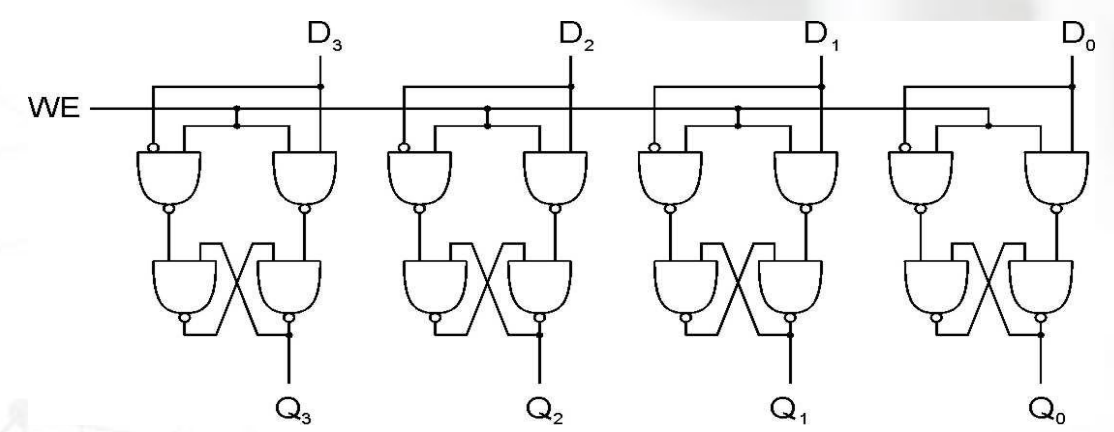
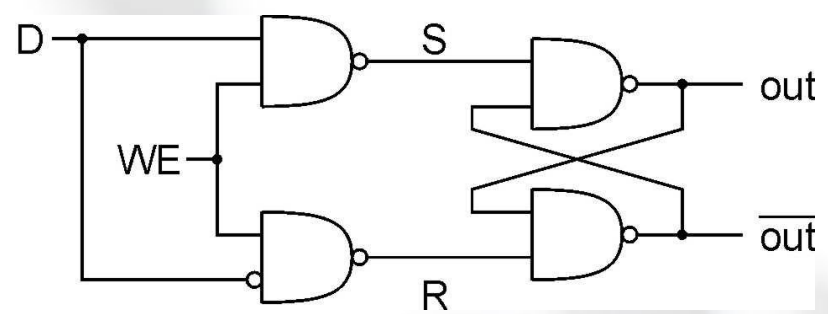
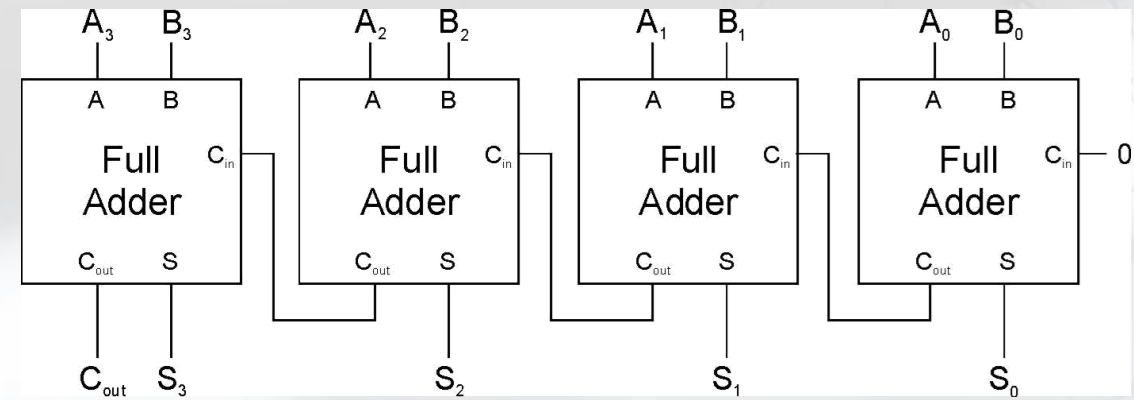
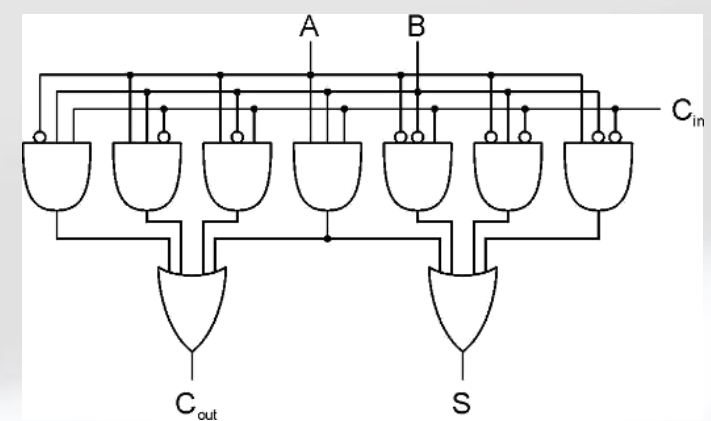
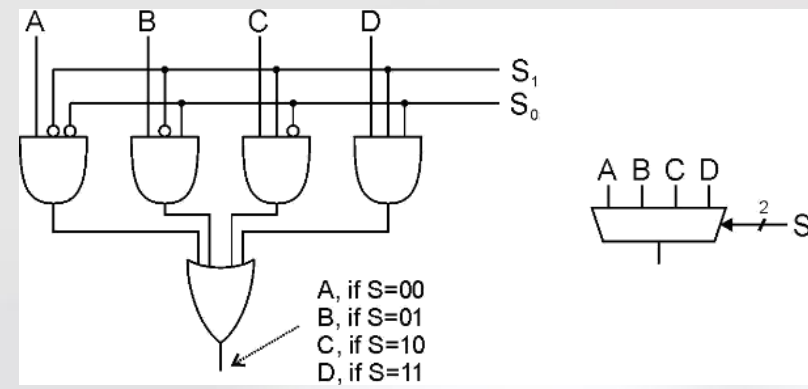
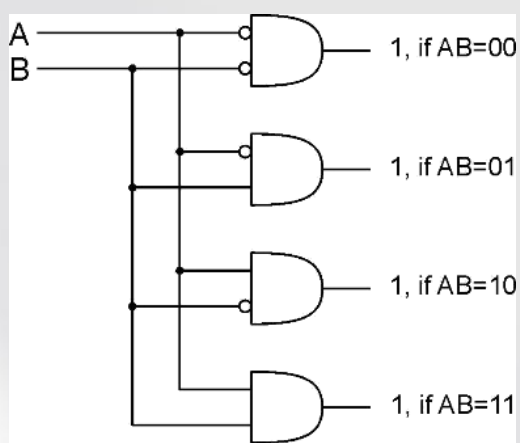
1 Review

2 Assembly Language Overview

3 Assembly Process

4 Summary

Review: The Transistor & Basic Logical Structure

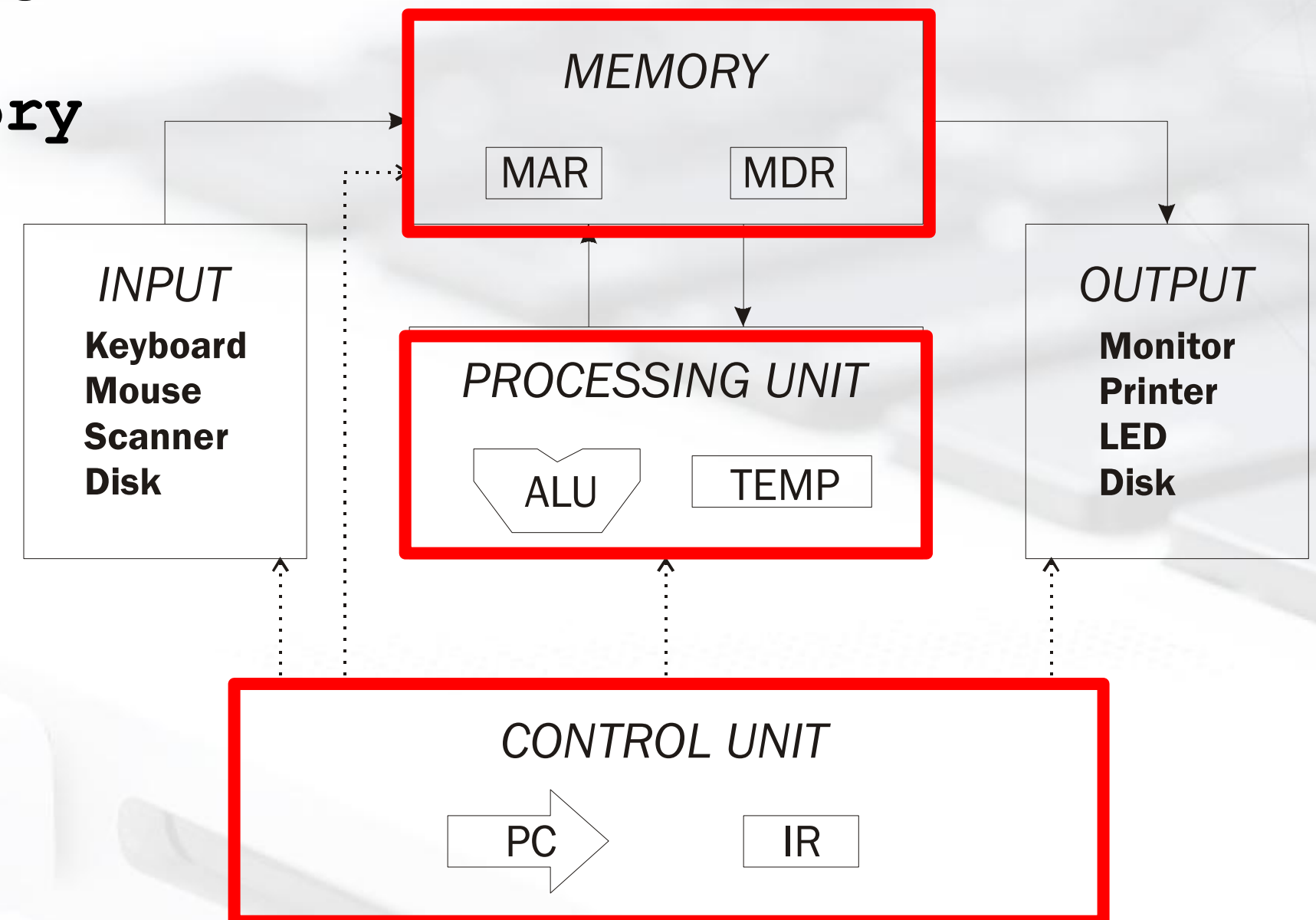


Review: Von Neumann Model



■ So far, we've learned how to:

- compute with values in registers
- load data from memory to registers
- store data from registers to memory





运算指令																
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD	0	0	0	1	DR			SR1			0	0	0	SR2		
ADD	0	0	0	1	DR			SR1			1	Imm5				
AND	0	1	0	1	DR			SR1			0	0	0	SR2		
AND	0	1	0	1	DR			SR1			1	Imm5				
NOT	1	0	0	1	DR			SR1			1	1	1	1	1	1
Reserved	1	1	0	1												

控制指令																
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR	0	0	0	0	n	z	p	PCOffset9								
JSR	0	1	0	0	1	PCOffset11										
JSRR	0	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	
RTI	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
JMP	1	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	
RET	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	
TRAP	1	1	1	1	0	0	0	0	TrapVector8							

移动数据指令

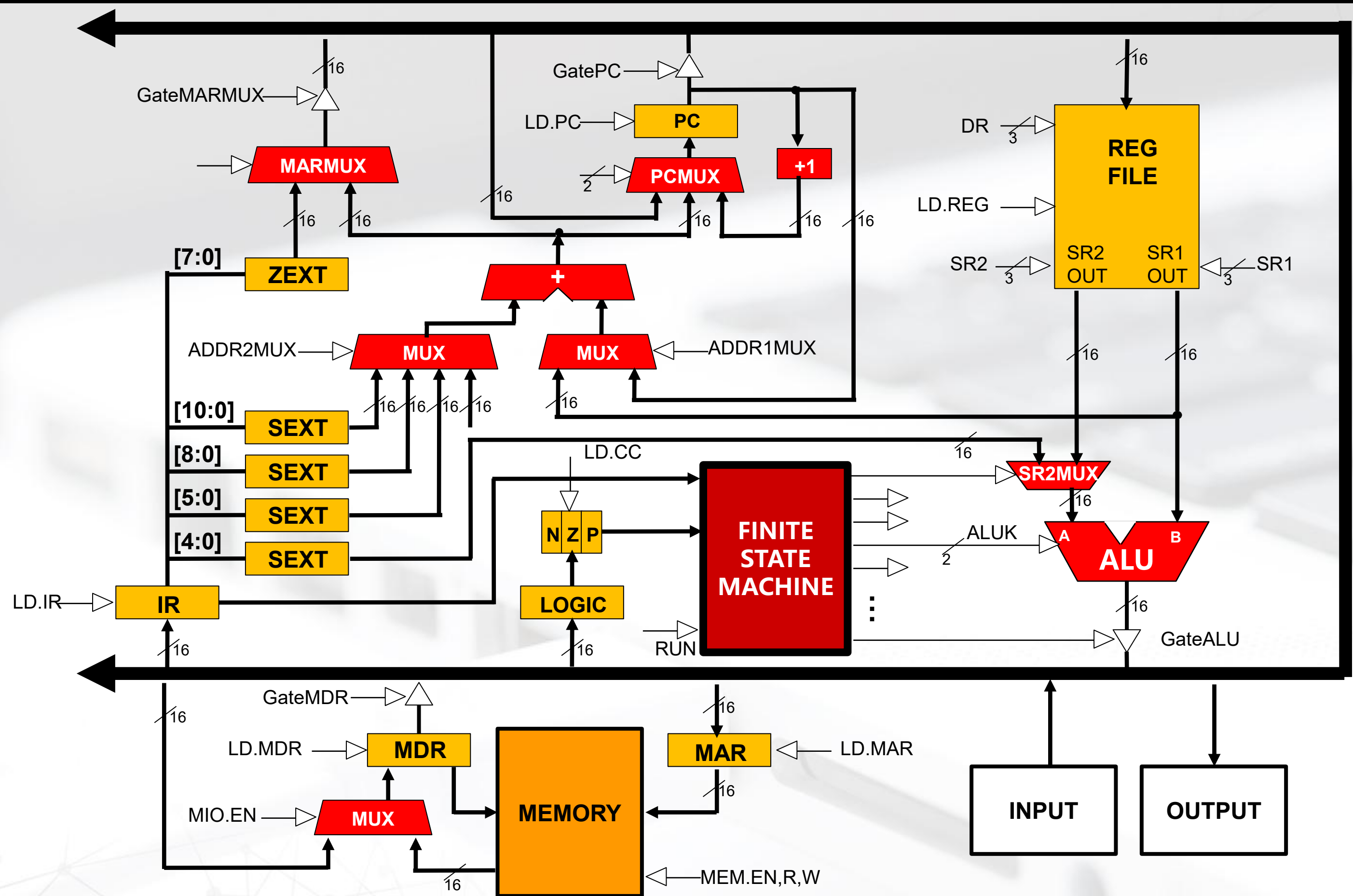
取数指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LD	0	0	1	0	DR		PCOffset9									
LDR	0	1	1	0	DR		BaseR		PCOffset6							
LDI	1	0	1	0	DR		PCOffset9									
LEA	1	1	1	0	DR		PCOffset9									

存数指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ST	0	0	1	1	SR		PCOffset9									
STR	0	1	1	1	SR		BaseR		PCOffset6							
STI	1	0	1	1	SR		PCOffset9									

Review: The Data Path(von Neumann Model)



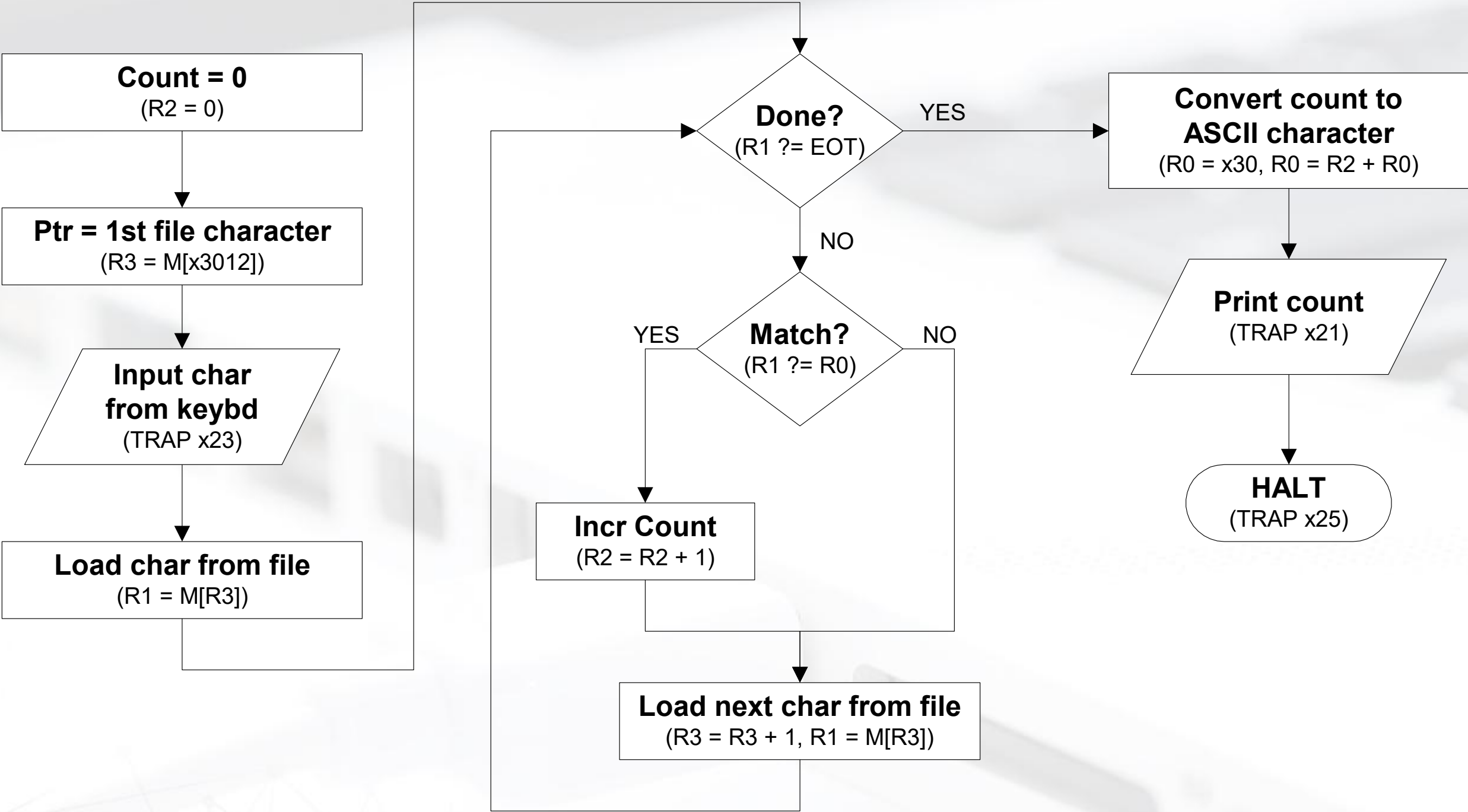
Counting the occurrences of a character in a file

- Program begins at location **x3000**
- Read character from keyboard
- Load each character from a “file”
 - File is a sequence of memory locations
 - Starting address of file is stored in the memory location immediately after the program
- If file character equals input character, increment counter
- End of file is indicated by a special ASCII value: **EOT (x04)**
- At the end, print the number of characters and halt
(assume there will be less than 10 occurrences of the character)

Sample Program

Remember this?

- Count the occurrences of a character in a file.



Program (1 of 2)

Address	Instruction													Comments	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	$R2 \leftarrow 0$ (counter) AND R2,R2, #0
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	$R3 \leftarrow M[x3012]$ (ptr) LD R3, x3012 (LD R3, PTR)
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	Input to R0 (TRAP x23) TRAP x23 (GETC)
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	$R1 \leftarrow M[R3]$ LDR R1, R3, #0
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	0	$R4 \leftarrow R1 - 4$ (EOT) ADD R4,R1, #-4
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	If Z, goto x300E BRz x300E (BRz OUTPUT)
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	$R1 \leftarrow NOT R1$ NOT R1,R1
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	$R1 \leftarrow R1 + 1$ ADD R1,R1,#1
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	$R1 \leftarrow R1 + R0$ ADD R1,R1,R0
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	If N or P, goto x300B BRnp x300B (BRnp GETCHAR)

Program (2 of 2)



Address	Instruction															Comments	
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	$R2 \leftarrow R2 + 1$ ADD R2,R2,#1
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	$R3 \leftarrow R3 + 1$ ADD R3,R3,#1
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	$R1 \leftarrow M[R3]$ LDR R1,R3,#0
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	Goto x3004 BRnzp x3004 (BRnzp TEST)
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	$R0 \leftarrow M[x3013]$ LD R0,x3013 (LD R0, ASCII)
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	$R0 \leftarrow R0 + R2$ ADD R0,R0,R2
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	Print R0 TRAP x21 (OUT)
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT TRAP x25 (HALT)
x3012	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	Starting Address of File (X9000)
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	ASCII x30 ('0')

1

Review

2

Assembly Language Overview

3

Assembly Process

4

Summary

A LC-3 Program



X4101	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0
X4102	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0
X4103	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1
X4104	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0
X4105	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0
X4106	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0
X4107	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1
X4108	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1
X4109	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0
X410A	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1
X410B	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1
X410C	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1
X410D	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0
X410E	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0
X410F	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0
X4110	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0
X4101	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1
X4102	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1
X4103	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
X4104	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

X8001	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0
X8002	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0
X8003	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	1
X8004	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0
X8005	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0
X8006	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	1
X8007	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1
X8008	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	1
X8009	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0
X800A	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
X800B	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
X800C	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
X800D	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0
X800E	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	1
X800F	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	0	0
X8010	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0
X8011	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
X8012	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0
X8013	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	1	0
X8014	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
X8015	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
X8016	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
X8017	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1	0
X8018	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

Human-Readable Machine Language

■ Computers like ones and zeros...

0001110010000110

■ Humans like symbols...

ADD R6,R2,R6 ; increment index reg.

or

C = a + b;

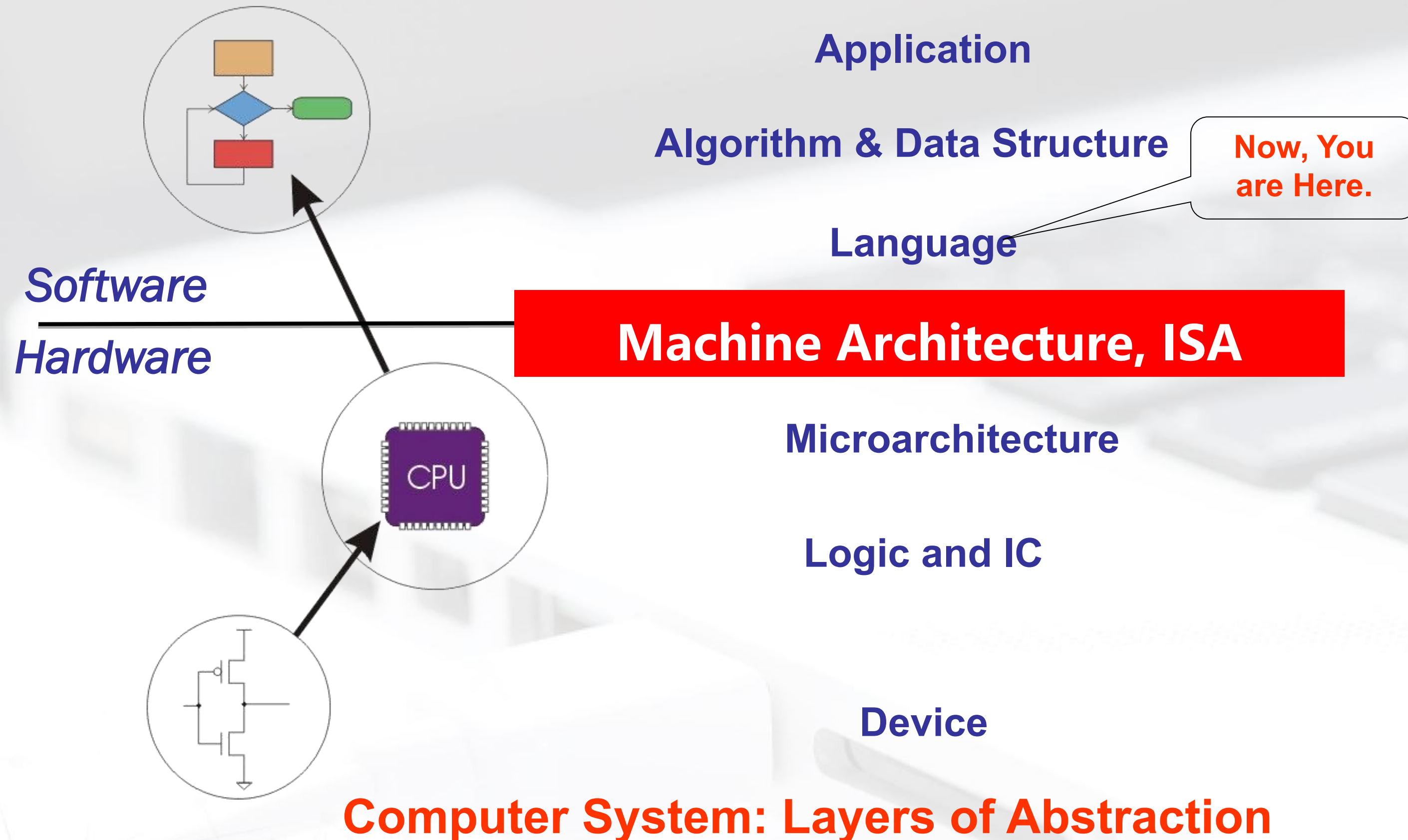
■ Assembler is a program that turns symbols into machine instructions.

- **ISA-specific: close correspondence between symbols and instruction set**

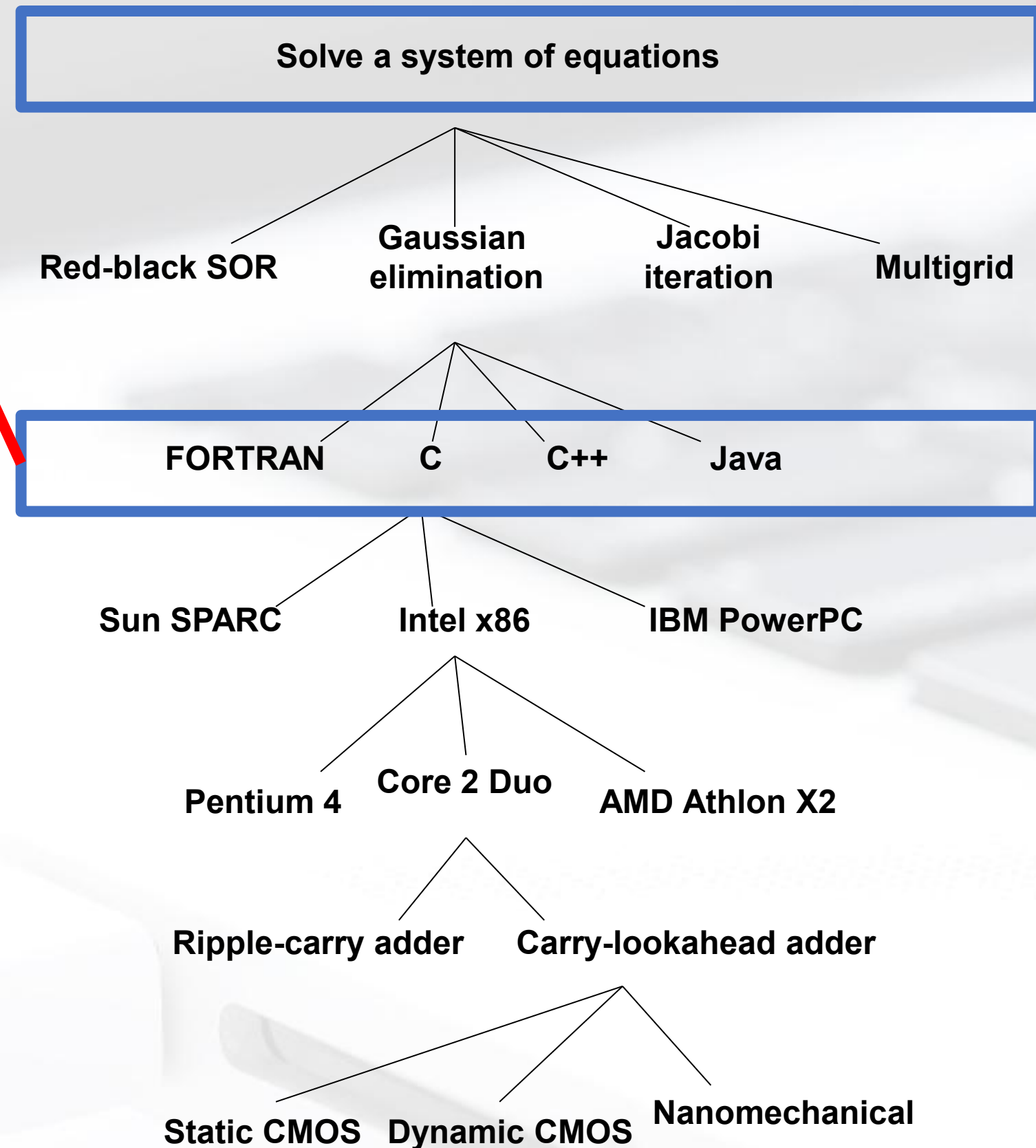
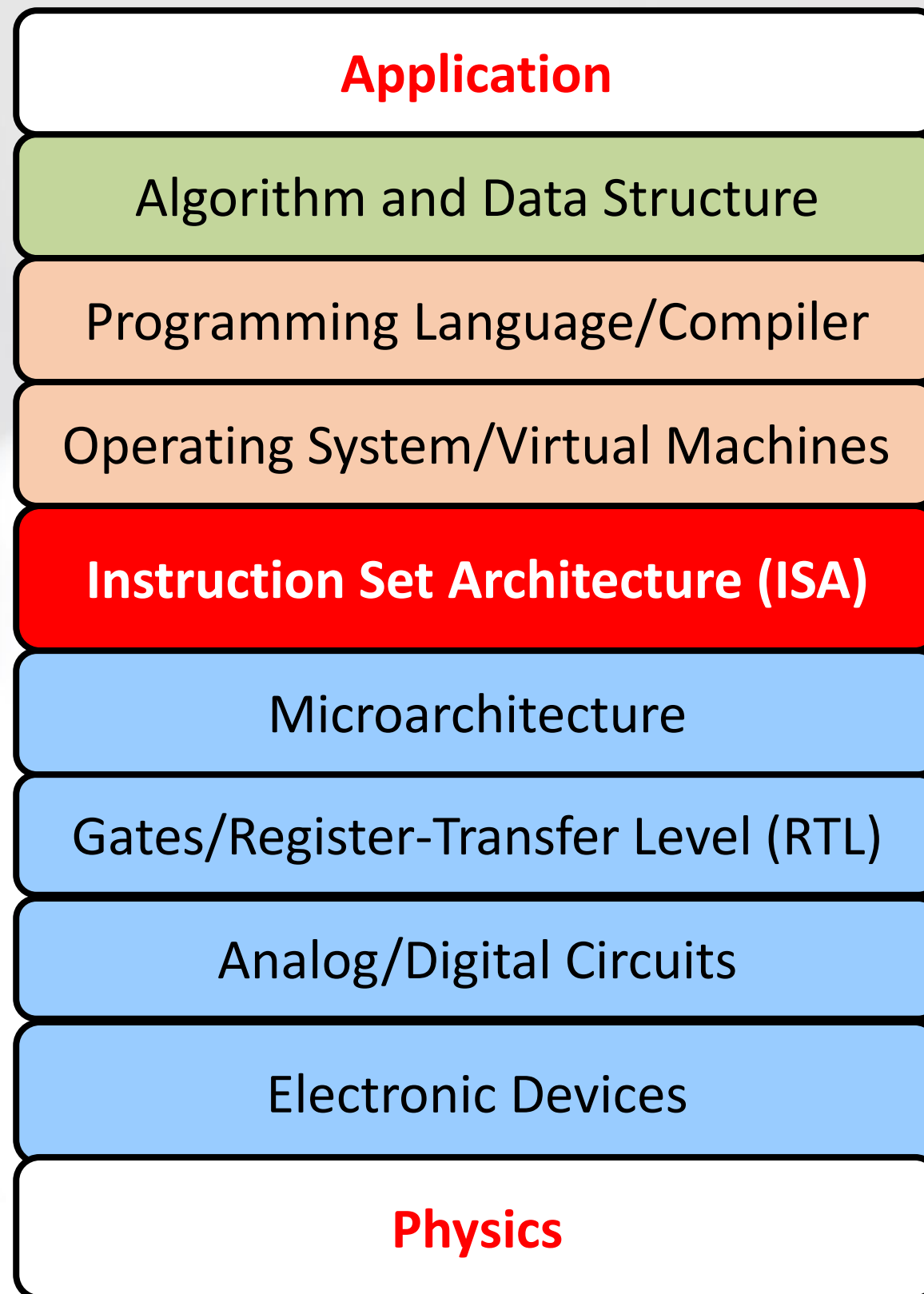
- mnemonics for opcodes
- labels for memory locations

- **additional operations for allocating storage and initializing data**

Great Idea #4: Software and Hardware Co-design



Great Idea #3: Abstraction Helps Us Manage Complexity



An Assembly Language Program

```
01 ;
02 ; Program to multiply a number by the constant 6
03 ;
04     .ORIG    x3050
05     LD      R1, SIX
06     LD      R2, NUMBER
07     AND     R3, R3, #0    ; Clear R3.  It will
08                               ; contain the product.
09 ; The inner loop
0A ;
0B AGAIN ADD    R3, R3, R2
0C     ADD    R1, R1, #-1    ; R1 keeps track of
0D     BRp    AGAIN        ; the iteration.
0E ;
0F     HALT
10 ;
11 NUMBER .BLKW  1
12 SIX    .FILL  x0006
13 ;
        .END
```


LC-3 Assembly Language Syntax

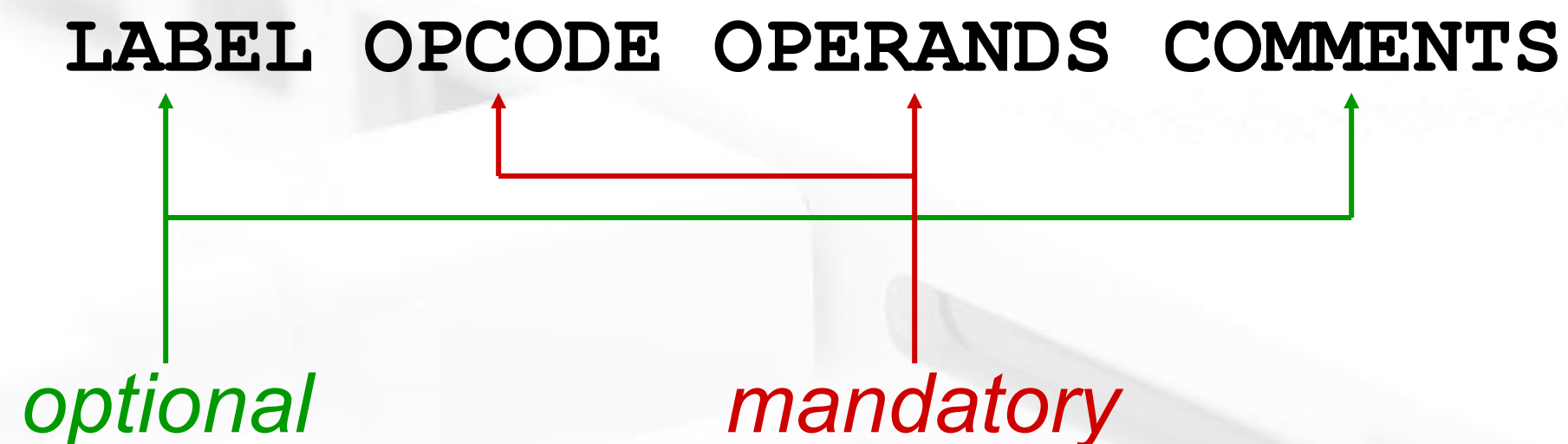
■ Each line of a program is one of the following:

- an instruction
- an assembler directive (or pseudo-op)
- a comment

■ Whitespace (between symbols) and case are ignored.

■ Comments (beginning with “;”) are also ignored.

■ An instruction has the following format:



Opcodes and Operands

■ Opcodes

- reserved symbols that correspond to LC-3 instructions
- listed in Appendix A
 - ex: ADD, AND, LD, LDR, ...

■ Operands

- registers -- specified by **Rn**, where n is the register number
- numbers -- indicated by **# (decimal)** or **x (hex)** or **b (binary)**
- label -- symbolic name of memory location
- separated by comma
- number, order, and type correspond to instruction format

```
ADD  R1, R1, R3
ADD  R1, R1, #3
LD   R6, NUMBER
BRz  LOOP
```

Labels and Comments

■ Label

- placed at the beginning of the line
- assigns a symbolic name to the address corresponding to line

```
LOOP ADD R1, R1, #-1  
      BRp LOOP
```

■ Comment

- anything after a semicolon is a comment
- ignored by assembler
- used by humans to document/understand programs
- tips for useful comments:
 - avoid restating the obvious, as “decrement R1”
 - provide additional insight, as in “accumulate product in R6”
 - use comments to separate pieces of program

Assembler Directives

■ Pseudo-operations

- do not refer to operations executed by program
- used by assembler
- look like instruction, but "opcode" starts with dot

<i>Opcode</i>	<i>Operand</i>	<i>Meaning</i>
.ORIG	address	starting address of program
.END		end of program
.BLKW	n	allocate n words of storage
.FILL	value	allocate one word, initialize with a value
.STRINGZ	n-character string	allocate n+1 locations, initialize w/characters and null terminator

Example

```
.ORIG    x3010  
HELLO    .STRINGZ    " Hello, World! "
```

```
x3010:  x0048  
x3011:  x0065  
x3012:  x006C  
x3013:  x006C  
x3014:  x006F  
x3015:  x002C  
x3016:  x0020  
x3017:  x0057  
x3018:  x006F  
x3019:  x0072  
x301A:  x006C  
x301B:  x0064  
x301C:  x0021  
x301D:  x0000
```

Trap Codes

- LC-3 assembler provides “pseudo-instructions” for each trap code, so you don’t have to remember them.

<i>Code</i>	<i>Equivalent</i>	<i>Description</i>
HALT	TRAP x25	Halt execution and print message to console.
IN	TRAP x23	Print prompt on console, read (and echo) one character from keybd. Character stored in R0[7:0].
OUT	TRAP x21	Write one character (in R0[7:0]) to console.
GETC	TRAP x20	Read one character from keyboard. Character stored in R0[7:0].
PUTS	TRAP x22	Write null-terminated string to console. Address of string is in R0.

Style Guidelines

■ Use the following style guidelines to improve the readability and understandability of your programs:

1. Provide a program header, with author's name, date, etc., and purpose of program.
2. Start labels, opcode, operands, and comments in same column for each line. (Unless entire line is a comment.)
3. Use comments to explain what each register does.
4. Give explanatory comment for most instructions.
5. Use meaningful symbolic names.
 - Mixed upper and lower case for readability.
 - ASCIItoBinary, InputRoutine, SaveR1
6. Provide comments between program sections.
7. Each line must fit on the page -- no wraparound or truncations.
 - Long statements split in aesthetically pleasing manner.

Char Count in Assembly Language (1 of 3)

```
01 ;
02 ; Program to count occurrences of a character in a file.
03 ; Character to be input from the keyboard.
04 ; Result to be displayed on the monitor.
05 ; Program only works if no more than 9 occurrences are
06 ; found.
07 ;
08 ; Initialization
09 ;
0A .ORIG x3000
0B AND R2, R2, #0 ; R2 is counter, initially 0
0C LD R3, PTR ; R3 is pointer to characters
0D GETC ; TRAP x23
0E ; R0 gets character input
0F LDR R1, R3, #0 ; R1 gets first character
10 ;
11 ; Test character for end of file
12 ;
13
14 TEST ADD R4, R1, #-4 ; Test for EOT (ASCII x04)
15 BRz OUTPUT ; If done, prepare the output
```


Char Count in Assembly Language (2 of 3)

```
16 ;
17 ; Test character for match.  If a match, increment count.
18 ;
19     NOT    R1, R1
1A     ADD    R1, R1, #1 ; R1 <-- -R1
1B     ADD    R1, R1, R0 ; R1 <-- R0 - R1.  if R1=0, a match!
1C     BRnp   GETCHAR    ; If no match, do not increment
1D     ADD    R2, R2, #1
1E ;
1F ; Get next character from file.
20 ;
21 GETCHAR ADD    R3, R3, #1 ; Point to next character.
22         LDR    R1, R3, #0 ; R1 gets next char to test
23         BRnzp  TEST
24 ;
25 ; Output the count.
26 ;
27 OUTPUT LD      R0, ASCII ; Load the ASCII template
28         ADD    R0, R0, R2 ; Covert binary count to ASCII
29         OUT                      ; TRAP x21
2A                                ; ASCII code in R0 is displayed.
2B         HALT                  ; TRAP x25, Halt machine
```

Char Count in Assembly Language (3 of 3)

```
2C ;  
2D ; Storage for pointer and ASCII template  
2E ;  
2F ASCII .FILL    x0030      ; ASCII code of number '0'  
30 PTR    .FILL    x9000      ; pointer to the first character  
31      .END
```

1

Review

2

Assembly Language Overview

3

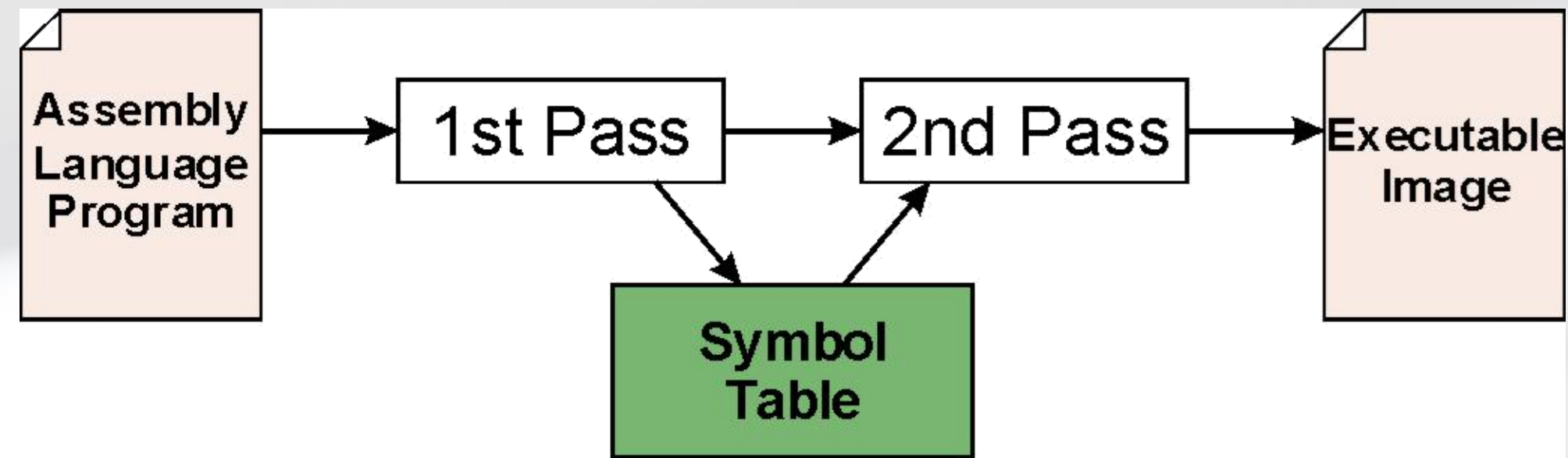
Assembly Process

4

Summary

Assembly Process

- Convert assembly language file (.asm) into an executable file (.obj) for the LC-3 simulator.



■ First Pass:

- scan program file
- find all labels and calculate the corresponding addresses;
this is called the symbol table

■ Second Pass:

- convert instructions to machine language, using information from symbol table

First Pass: Constructing the Symbol Table

1. Find the `.ORIG` statement, which tells us the address of the first instruction.

- Initialize location counter (LC), which keeps track of the current instruction.

2. For each non-empty line in the program:

a) If line contains a label, add label and LC to symbol table.

b) Increment LC.

- NOTE: If statement is `.BLKW` or `.STRINGZ`, increment LC by the number of words allocated.

3. Stop when `.END` statement is reached.

■ NOTE: A line that contains only a comment is considered an empty line.

■ Construct the symbol table for the program in Figure 7.2

```
; Initialization
;
        .ORIG      x3000
        AND        R2,R2,#0
        LD         R3,PTR
        TRAP       x23
        LDR.       R1,R3,#0
;
; Test char. For end of file
;
TEST    ADD        R4,R1,#-4
        BRz        OUTPUT
;
; Test char. For match.
;
        NOT        R1,R1
        ADD        R1,R1,#1
        ADD        R1,R1,R0
        BRnp       GETCHAR
        ADD        R2,R2,#1
```

```
;
; Get next char. from the file
;
GETCHAR ADD        R3,R3,#1
        LDR        R1,R3,#0
        BRnp       TEST
;
; Output the count
;
OUTPUT  LD         R0,ASCII
        ADD        R0,R0,R2
        TRAP       x21
        TRAP       x25
;
; Storage for pointer and ASCII temp.
;
ASCII   .FILL      x0030
PTR     .FILL      x4000
        .END
```



■ Construct the symbol table for the program in Figure 7.2

Symbol	Address
TEST	x3004
GETCHAR	x300B
OUTPUT	x300E
ASCII	x3012
PTR	x3013

Second Pass: Generating Machine Language

■ For each executable assembly language statement, generate the corresponding machine language instruction.

- If operand is a label, look up the address from the symbol table.

■ Potential problems:

- Improper number or type of arguments

- ex: NOT R1, #7
 - ADD R1, R2
 - ADD R3, R3, NUMBER

- Immediate argument too large

- ex: ADD R1, R2, #1023

- Address (associated with label) not on the same page

- can't use direct addressing mode

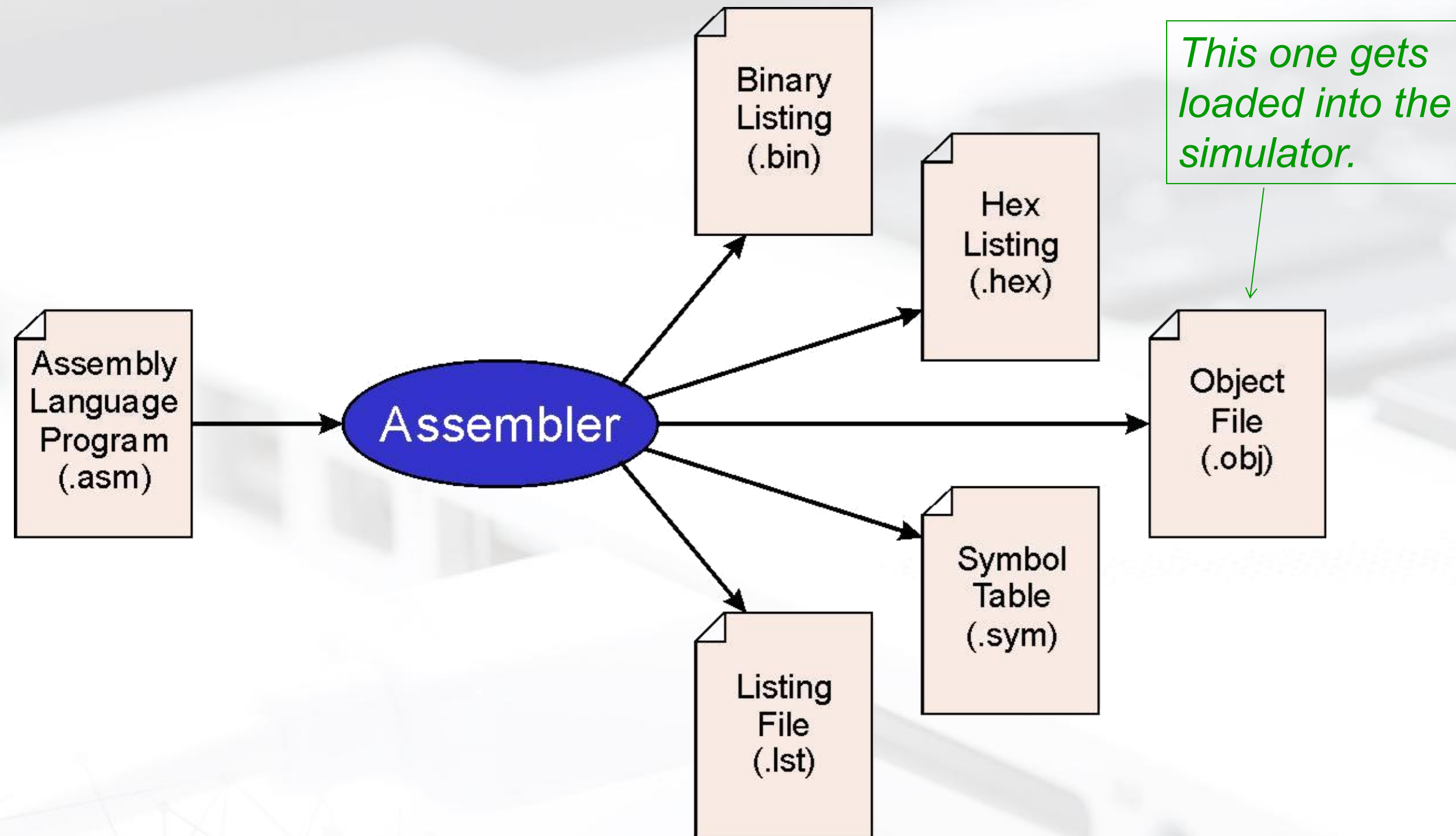
Practice

- Using the symbol table constructed earlier, translate these statements into LC-3 machine language.
- *(Assume all addresses are on the current page.)*

	Statement	Machine Language
x3001	LD R3 , PTR	
x3003	LDR R1 , R3 , #0	
x3004	ADD R4 , R1 , #-4	
x300D	BRnp GETCHAR	

LC-3 Assembler

- Using “assemble” (Unix) or LC3 Edit (Windows), generates several different output files.



Object File Format

■ LC-3 object file contains

- Starting address (location where program must be loaded), followed by...
- Machine instructions

■ Example

- Beginning of "count character" object file looks like this:

0011000000000000	← .ORIG x3000
0101010010100000	← AND R2, R2, #0
0010011000010100	← LD R3, PTR
1111000000100011	← TRAP x23
·	
·	
·	

Multiple Object Files

■ An object file is not necessarily a complete program.

- system-provided library routines
- code blocks written by multiple developers

■ For LC-3, can load multiple object files into memory, then start executing at a desired address.

- system routines, such as keyboard input, are loaded automatically
 - loaded into “system memory,” below x1000
 - by convention, user code should be loaded between x3000 and xCFFF
- each object file includes a starting address
- be careful not to load overlapping object files

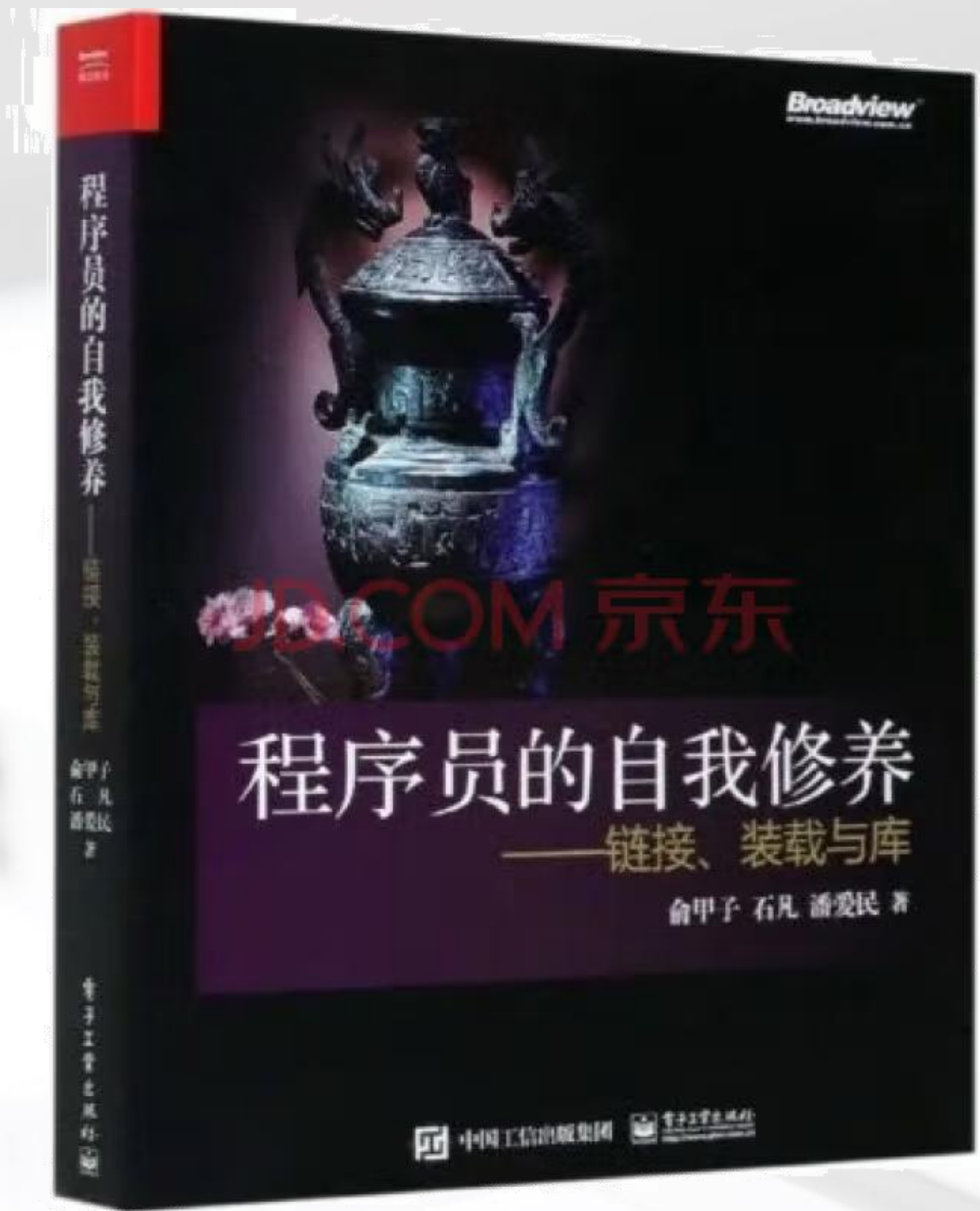
Linking and Loading

■ **Loading** is the process of copying an executable image into memory.

- more sophisticated loaders are able to relocate images to fit into available memory
- must readjust branch targets, load/store addresses

■ **Linking** is the process of resolving symbols between independent object files.

- suppose we define a symbol in one module, and want to use it in another
- some notation, such as `.EXTERNAL`, is used to tell assembler that a symbol is defined in another module
- linker will search symbol tables of other modules to resolve symbols and complete code generation before loading



1

Review

2

Assembly Language Overview

3

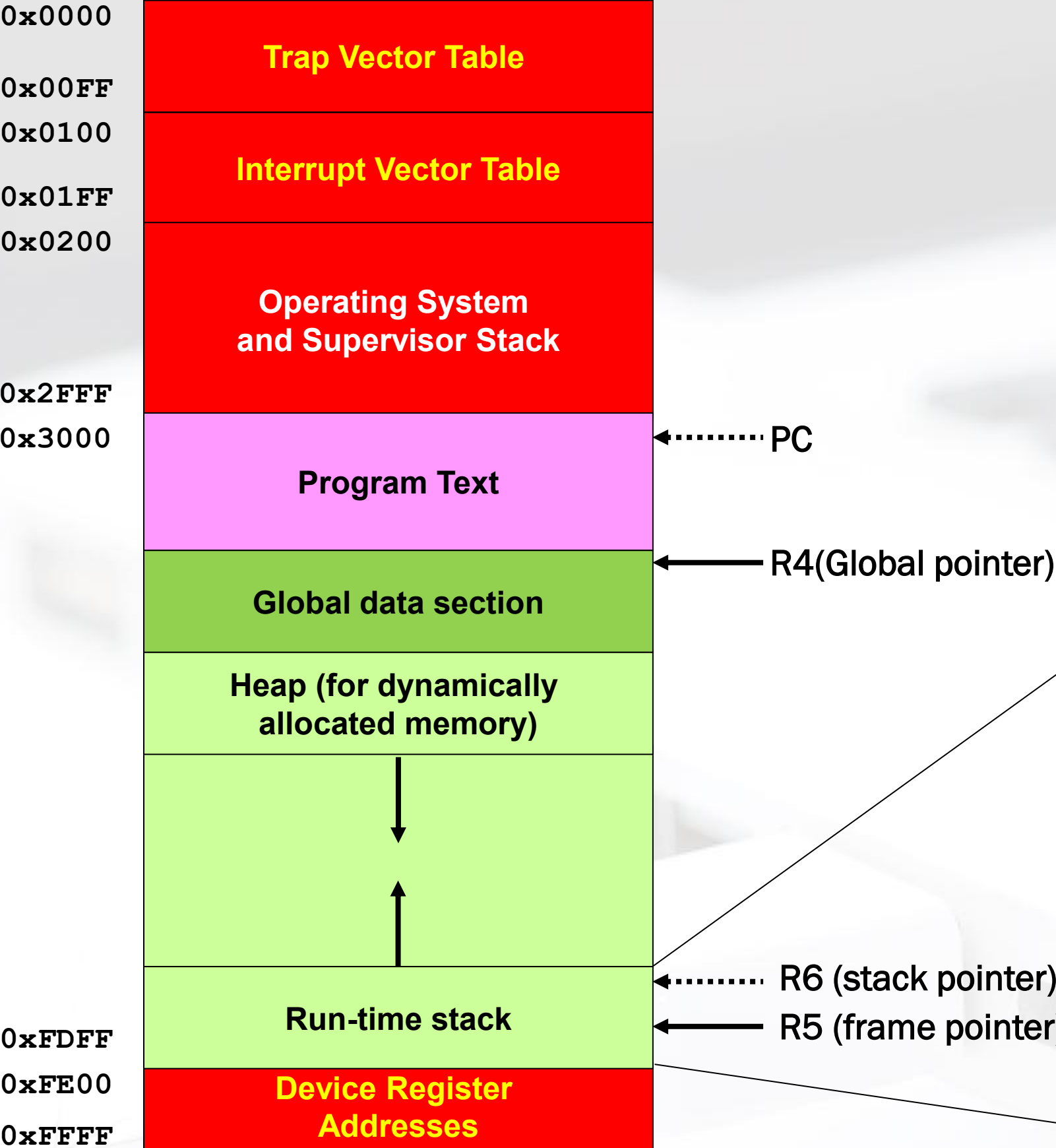
Assembly Process

4

Summary

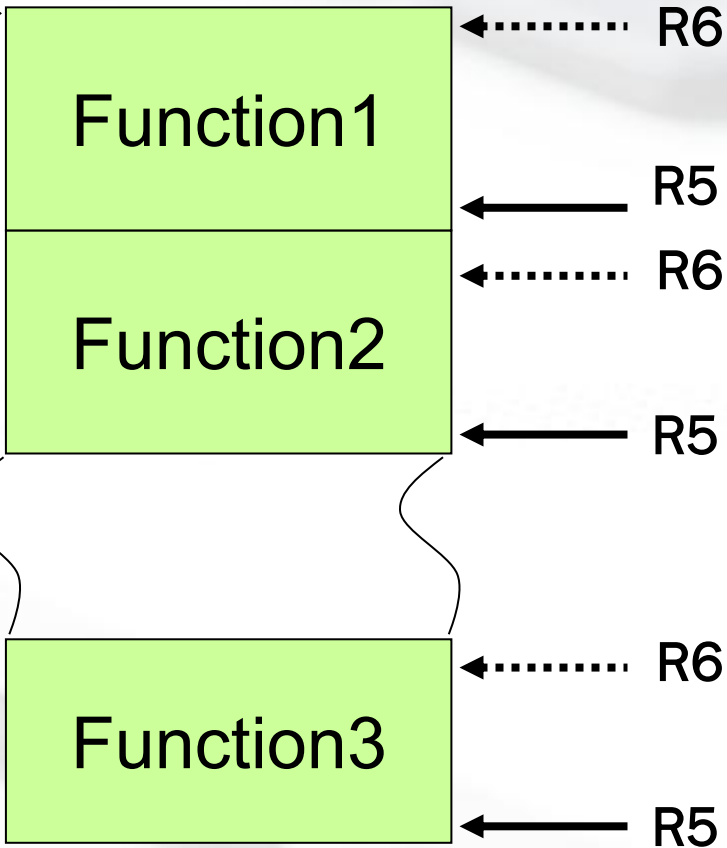


Memory map of the LC-3



Memory

- address space: 2^{16} locations (16-bit addresses)
- addressability: 16 bits



Exercise 1

7.2 An LC-3 assembly language program contains the instruction:

```
ASCII  LD  R1, ASCII
```

The symbol table entry for ASCII is x4F08. If this instruction is executed during the running of the program, what will be contained in R1 immediately after the instruction is executed?

7.10 The following program fragment has an error in it. Identify the error and explain how to fix it.

```
                ADD      R3, R3, #30
                ST       R3, A
                HALT
A                .FILL   #0
```

Will this error be detected when this code is assembled or when this code is run on the LC-3?

Exercise 3

7.13 The following program adds the values stored in memory locations A, B, and C and stores the result into memory. There are two errors in the code. For each, describe the error and indicate whether it will be detected at assembly time or at run time.

```
Line No.
1          .ORIG x3000
2      ONE  LD  R0, A
3          ADD R1, R1, R0
4      TWO  LD  R0, B
5          ADD R1, R1, R0
6      THREE LD  R0, C
7          ADD R1, R1, R0
8          ST  R1, SUM
9          TRAP x25
10     A    .FILL x0001
11     B    .FILL x0002
12     C    .FILL x0003
13     D    .FILL x0004
14          .END
```


7.24 We want the following program fragment to shift R3 to the left by four bits, but it has an error in it. Identify the error and explain how to fix it.

```
        .ORIG  x3000
        AND    R2, R2, #0
        ADD    R2, R2, #4
LOOP    BRz    DONE
        ADD    R2, R2, #-1
        ADD    R3, R3, R3
        BR     LOOP
DONE    HALT
        .END
```

Exercise 5

6.18 The LC-3 has no Divide instruction. A programmer needing to divide two numbers would have to write a routine to handle it. Show the systematic decomposition of the process of dividing two positive integers. Write an LC-3 machine language program starting at location x3000 that divides the number in memory location x4000 by the number in memory location x4001 and stores the quotient at x5000 and the remainder at x5001.

Exercise 6

★5.50 Three instructions all construct an address by sign-extending the low nine bits of the instruction and adding it to the incremented PC.

The Conditional Branch

0	0	0	0	1	1	1	x	x	x	x	x	x	x	x	x
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The Load Effective Address

1	1	1	0	1	1	1	x	x	x	x	x	x	x	x	x
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The LD Instruction

0	0	1	0	1	1	1	x	x	x	x	x	x	x	x	x
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The xxxxxxxxxx represents the nine-bit offset that is sign-extended.

Where does the LC-3 microarchitecture put the result of adding the nine-bit sign-extended offset to the incremented PC?

Exercise 7

★6.23 The PC is loaded with x3000, and the instruction at address x3000 is executed. In fact, execution continues and four more instructions are executed. The table below contains the contents of various registers at the end of execution for each of the five (total) instructions. Your job: Complete the table.

	PC	MAR	MDR	IR	R0	R1
Before execution starts	x3000	—	—	—	x0000	x0000
After the first finishes			xB333	x2005		
After the 2nd finishes				x0601		
After the 3rd finishes			x1---			x0001
After the 4th finishes			x1---		x6666	
After the 5th finishes				x0BFC		

Let's start execution again, starting with PC = x3000. First, we re-initialize R0 and R1 to 0, and set a breakpoint at x3004. We press RUN eleven times, and each time the program executes until the breakpoint. What are the final values of R0 and R1?

7.34 It is often useful to find the midpoint between two values. For this problem, assume A and B are both even numbers, and A is less than B. For example, if $A = 2$ and $B = 8$, the midpoint is 5. The following program finds the midpoint of two even numbers A and B by continually incrementing the smaller number and decrementing the larger number. You can assume that A and B have been loaded with values before this program starts execution.
Your job: Insert the missing instructions.

```

                .ORIG  x3000
LD             R0,A
LD             R1,B
X              ----- (a)
                ----- (b)
ADD            R2,R2,R1
                ----- (c)
ADD            R1,R1,#-1
                ----- (d)
BRnzp         X
DONE          ST      R1,C
              TRAP    x25
A             .BLKW   1
B             .BLKW   1
C             .BLKW   1
              .END

```

★7.29 An LC-3 program is executing on the LC-3 simulator when a breakpoint is encountered, and the simulator stops. At that point, **the contents of several registers are as shown in the first row of the table**. After the run button is subsequently pushed, the next four instructions that are executed, none of which are an STI or LDI, produce the values shown in the table, **two rows of the table per instruction executed**. The first row of each pair shows the contents after the fetch phase of the corresponding instruction, and the second row of each pair after that instruction completes.

Note that some values are missing and are presented by letters A, B, C, D, E, F, G, H, I, and J.

PC	MAR	MDR	IR	R0	R1	R2	R3	R4	R5	R6	R7
x1800	x7FFF	x2211	xBFFE	x31FF	x2233	x5177	x3211	x21FF	x5233	x3177	x2211
A	x1800	B	B	x31FF	x2233	x5177	x3211	x21FF	x5233	x3177	x2211
A	x1800	B	B	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
D	A	E	E	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
D	F	G	E	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
H	D	I	I	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
F	D	I	I	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
A	F	J	J	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
A	F	J	J	x31FF	x2233	x5177	x3211	x223A	C	x3177	x2211

Your job: Determine the values of A, B, C, D, E, F, G, H, I, and J. Note that some of the values may be identical.

A	B	C	D	E

F	G	H	I	J
			x 'F'	

LC-3 Data Path

